Быстрая сортировка

Выполнил студент 211 группы факультета КНиИТ
Кочергин Антон

Основаная информация

- Сложность: $O(n \log n)$ в среднем, $O(n^2)$ в худшем случае (если массив уже отсортирован или содержит много одинаковых элементов).
- Одна из самых быстрых сортировок, когда ничего не известно про массивы
- Легко воспроизводится на разных языках

Основная идея алгоритма

- На очередном шаге выбирается опорный элемент — им может быть любой элемент массива
- Все остальные элементы массива сравниваются с опорным и те, которые меньше него, становятся слева от него, а которые больше или равны — справа
- Для двух блоков массива (меньше опорного, и больше либо равно опорному) производится точно так же с первого шага - появляется рекурсия

Вход: A — массив, L — левая граница массива, R— правая граница массива
Выход: Отсортированный массив
начало алгоритма

- $i = L$;
- $j = R$;
- выбирается опорный элемент;

цикл пока $i <= j$ **выполнять**

- **цикл пока** $a[i] <$ *опорного* **выполнять**
  - увеличиваем i;
- **цикл пока** $a[j] >$ *опорного* **выполнять**
  - уменьшаем j;
- **если** $i <= j$ **то**
  - меняем местами $a[i]$ и $a[j]$;
  - уменьшаем j;
  - увеличиваем i;

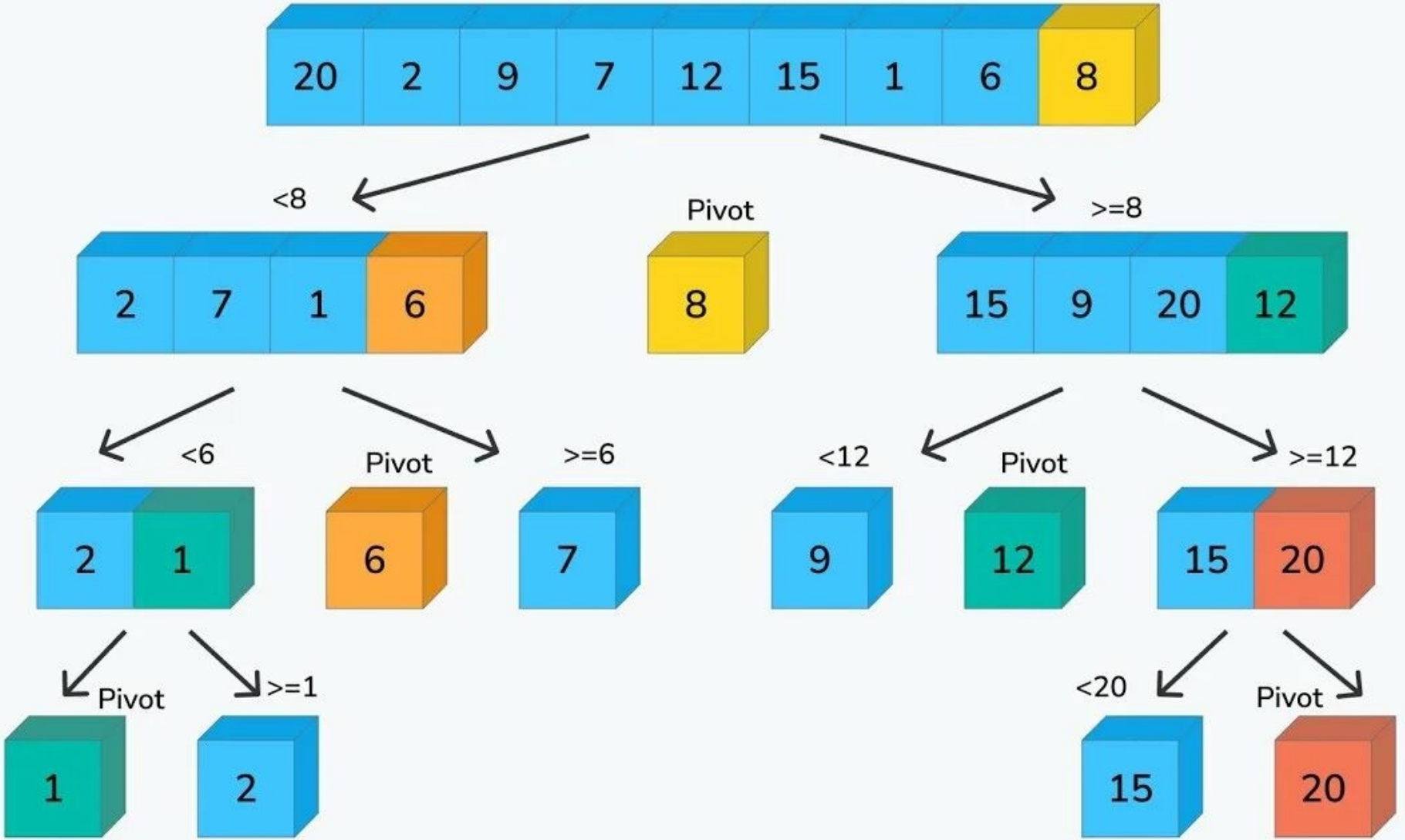
если *левый подмассив содержит больше одного элемента* **то**

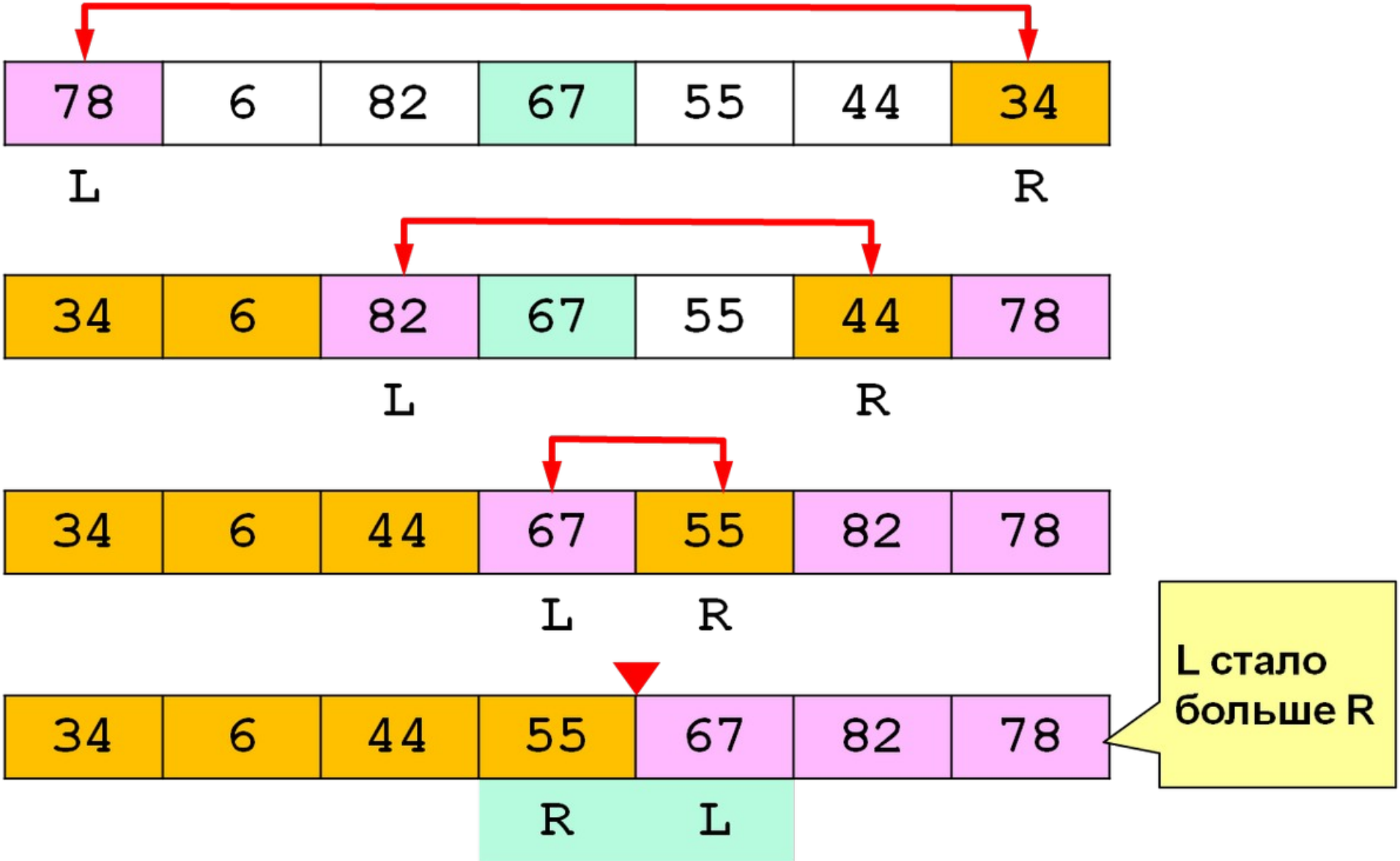
- вызываем функцию для $[L, j]$;

если *правый подмассив содержит больше одного элемента* **то**

- вызываем функцию для $[i, R]$;

конец алгоритма


20 2 9 7 12 15 1 6 8
<8
Pivot
>=8
2 7 1 6
8
15 9 20 12
<6
Pivot
>=6
<12
Pivot
>=12
2 1
6
7
9
12
15 20
Pivot
>=1
<20
Pivot
1
2
15
20


78 6 82 67 55 44 34
L R
34 6 82 67 55 44 78
L R
34 6 44 67 55 82 78
L R
34 6 44 55 67 82 78
R L
L стало больше R

Алгоритм на языке C++

```
void quickSort(int a[], int left, int right)
{
    if (left < right)
    {
        int p = a[(left + right) / 2];
        int i = left, j = right;
        while (i <= j)
        {
            while (a[i] < p)
                i++;
            while (a[j] > p)
                j--;
            if (i <= j)
            {
                swap(a[i], a[j]);
                i++;
                j--;
            }
        }
        quickSort(a, left, j);
        quickSort(a, i, right);
    }
}
```

Способы выбора опорного элемента

- Первый элемент
- Средний по индексу элемент
- Случайный элемент
- Последний элемент
- Медиана трёх

Способы выбора опорного элемента

- Первый элемент int p = a[left]
- Средний элемент int p = a[(left + right) / 2]
- Случайный элемент

  int random = left + rand() % (right — left +1);

  int p = a[random]
- Последний элемент int p = a[right]

Способы выбора опорного элемента

- Медиана трёх

```
int mid = (left + right) / 2;
vector<int> temp = {a[left] , a[mid], a[right]};
sort(temp.begin(),temp.end());
p = temp[1];
```

Оценка сложности

- Для лучшего и среднего случая — $O(n \log n)$
- Для худшего случая — $O(n^2)$

Сложность алгоритма сортировки может менять в зависимости от того, какой элемент будет выбран в качестве опорного